MP6: Primitive Disk Device Driver

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

Main: Nonblocking Disk - Completed
Bonus Option 1: Design of a thread-safe disk system - Completed
Bonus Option 2: Implementation of a thread-safe disk system - Completed
Bonus Option 3: Using Interrupts for Concurrency - Completed

System Design

The system is structured to support non-blocking disk operations inside the kernel by using a hybrid of cooperative scheduling and interrupt-driven I/O. At the center of the design is the `NonBlockingDisk` class, which derives from `SimpleDisk` and augments it with asynchronous read and write capabilities.

This design ensures that while one thread is waiting for a disk operation to complete, other threads can continue executing. Non-blocking behavior is achieved through:

- Maintaining a dedicated queue of threads that are blocked waiting for disk readiness.
- Installing an interrupt handler on IRQ 14 that resumes the next waiting thread when the disk signals completion of an I/O operation.
- Integrating with the system scheduler to yield the CPU whenever the disk is busy and to resume pending threads once the device becomes ready.

Additionally, when the `_THREAD_SYNCHRONIZATION_` macro is enabled, a lightweight concurrency-control mechanism is used to serialize disk access and prevent race conditions when multiple threads attempt to use the device concurrently. When the `_HANDLE_INTERRUPTS_` macro is enabled (Bonus Option 3), the disk driver further utilizes hardware interrupts to detect I/O completion instead of relying solely on polling. This interrupt-driven model significantly reduces CPU waste and provides true non-blocking I/O semantics.

Code Description

Files Modified: `scheduler.H`, `scheduler.C`, `nonblocking_disk.H`, `nonblocking_disk.C`

nonblocking_disk.C: Constructor(): Initializes the `NonBlockingDisk` class, which extends both `SimpleDisk` and `InterruptHandler`. It sets up the interrupt handler for IRQ 14 so the driver can be notified when a disk operation finishes. The constructor also creates a `concurrencyControl` object to ensure mutually exclusive access when multiple threads attempt to interact with the disk.

```
/*--------------------------------------------------------------------------*/
/* CONSTRUCTOR */
concurrencyControl* concur_control = nullptr;
extern Scheduler * SYSTEM_SCHEDULER;
/*--------------------------------------------------------------------------*/

NonBlockingDisk::NonBlockingDisk(unsigned int _size)
  : SimpleDisk(_size), InterruptHandler() {
    concur_control = new concurrencyControl();
    #ifdef _HANDLE_INTERRUPTS_
      InterruptHandler::register_handler(14, this);
    #endif
    Console::puts("[Disk] NonBlockingDisk constructed and interrupt handler registered (IRQ 14).\n");
}
```

Figure 1: Implementation of `NonBlockingDisk Constructor()`

nonblocking_disk.C: read(): This method overrides `SimpleDisk::read()` to implement non-blocking behavior. If `_THREAD_SYNCHRONIZATION_` is enabled, it first acquires the concurrency lock to ensure exclusive access to the disk. It then invokes the base class's `read()` to issue the actual disk command. After initiating the operation, it calls `wait_until_ready()` so the running thread can yield until the device becomes ready. Once the disk signals readiness, the thread resumes execution and releases the lock if it was previously acquired.

```
void NonBlockingDisk::read(unsigned long _block_no, unsigned char * _buf) {
  Console::puts("[Disk] Read requested on block: ");
  Console::puti(_block_no);
  Console::puts("\n");

  #ifdef _THREAD_SYNCHRONIZATION_
    Console::puts("[Disk] Acquiring lock for read...\n");
    concur_control->acquireLock();
  #endif

  SimpleDisk::read(_block_no, _buf);
  wait_until_ready();

  #ifdef _THREAD_SYNCHRONIZATION_
    Console::puts("[Disk] Releasing lock after read.\n");
    concur_control->releaseLock();
  #endif

  Console::puts("[Disk] Read completed for block: ");
  Console::puti(_block_no);
  Console::puts("\n");
}
```

Figure 2: Implementation of `read()`

nonblocking_disk.C: write(): This method mirrors the behavior of `read()`, overriding `SimpleDisk::write()` to provide non-blocking semantics. If `_THREAD_SYNCHRONIZATION_` is enabled, it begins by acquiring the concurrency lock to ensure exclusive access to the disk. It then invokes `SimpleDisk::write()` to issue the actual write command, followed by a call to `wait_until_ready()` so the calling thread can yield until the device finishes the operation. Once the disk signals completion—typically through the interrupt handler—the thread resumes and releases the lock if it had been acquired.

```
void NonBlockingDisk::write(unsigned long _block_no, unsigned char * _buf) {
  Console::puts("[Disk] Write requested on block: ");
  Console::puti(_block_no);
  Console::puts("\n");

  #ifdef _THREAD_SYNCHRONIZATION_
    Console::puts("[Disk] Acquiring lock for write...\n");
    concur_control->acquireLock();
  #endif

  SimpleDisk::write(_block_no, _buf);
  wait_until_ready();

  #ifdef _THREAD_SYNCHRONIZATION_
    Console::puts("[Disk] Releasing lock after write.\n");
    concur_control->releaseLock();
  #endif

  Console::puts("[Disk] Write completed for block: ");
  Console::puti(_block_no);
  Console::puts("\n");
}
```

Figure 3: Implementation of `write()`

nonblocking_disk.C: block_ready(): This helper method determines whether the disk is ready for a data transfer by checking the status register at port `0x1F7`. The device is treated as ready when the third status bit (mask `0x08`) is set. This routine is used by `wait_until_ready()` to poll the disk's readiness in both interrupt-driven and polling-based configurations.

```
bool NonBlockingDisk::block_ready()
{
  if(Machine::inportb(0x1F7) & 0x08 ==0 )
  {
    return false;
  }
  else
    return true;
}
```

Figure 4: Implementation of `block_ready()`

nonblocking_disk.C: wait_until_ready(): This method ensures that a thread waits for the disk to become ready after issuing a read or write request.

- When the `_HANDLE_INTERRUPTS_` macro is enabled, the current thread is placed into the internal wait queue and yields the CPU. It remains blocked until the disk triggers an interrupt, at which point the interrupt handler dequeues and resumes it.
- When interrupts are not enabled, the method repeatedly checks the disk's readiness status in a polling loop. On each iteration, the thread yields to allow other runnable threads to execute while waiting.

This design provides a unified interface that supports both interrupt-driven and polling-based synchronization, depending on the compilation configuration.

```
void NonBlockingDisk::wait_until_ready() {
  Console::puts("[Disk] Waiting for disk to become ready...\n");

  while (!block_ready()) {
    #ifdef _HANDLE_INTERRUPTS_
      Console::puts("[Disk] Enqueuing thread and yielding (interrupt mode)...\n");
      if (Machine::interrupts_enabled())
        Machine::disable_interrupts();
      thread_queue.enqueue(Thread::CurrentThread());
      if (!Machine::interrupts_enabled())
        Machine::enable_interrupts();
    #else
      Console::puts("[Disk] Disk not ready — yielding (polling mode)...\n");
      SYSTEM_SCHEDULER->resume(Thread::CurrentThread());
    #endif

    SYSTEM_SCHEDULER->yield();
  }
  Console::puts("[Disk] Disk is ready.\n");
}
```

Figure 5: Implementation of `wait_until_ready()`

nonblocking_disk.C: handle_interrupt() (shown in Bonus Option 3):

- When a disk interrupt is delivered, this handler removes the next waiting thread from the disk queue and resumes its execution through the scheduler.

Bonus Option 1: Design of a Thread-Safe Disk System

To enable safe concurrent disk access from multiple threads, the `NonBlockingDisk` class is extended with explicit locking to ensure thread safety. Without synchronization, simultaneous read or write operations can introduce race conditions whenever multiple threads access the disk at the same time. These issues may manifest as:

- Corrupted buffer contents if two threads attempt to write to the same block concurrently.
- Lost or inconsistent updates when a read is immediately followed by an overlapping write.
- Unpredictable behavior stemming from shared use of the `SimpleDisk` status register and internal buffers.

To address these hazards, the design incorporates a `concurrencyControl` lock object that enforces mutual exclusion on disk operations. This locking mechanism is activated whenever the `_THREAD_SYNCHRONIZATION_` macro is enabled. Both the `read()` and `write()` methods acquire the lock before issuing a disk command and release it once the operation completes, ensuring that only one thread interacts with the disk hardware at any given time.

```
#ifdef _THREAD_SYNCHRONIZATION_
    concur_control->acquireLock();
#endif
...
#ifdef _THREAD_SYNCHRONIZATION_
    concur_control->releaseLock();
#endif
```

This design integrates cleanly with both cooperative and interrupt-driven scheduling, without requiring any changes to the disk interface or kernel structure. It ensures safe concurrent access to the disk and prevents non-deterministic race conditions in multi-threaded environments.

Bonus Option 2: Implementation of a Thread-Safe Disk System

To realize the design outlined in Bonus Option 1, synchronization support was integrated directly into the `NonBlockingDisk` class. A global `concurrencyControl` object is used to enforce mutual exclusion during disk operations, ensuring that no two threads attempt to read or write through the disk interface simultaneously. This eliminates race conditions involving shared I/O ports, device registers, and disk buffers.

The thread-safety logic is conditionally enabled through the `_THREAD_SYNCHRONIZATION_` macro. When this macro is defined, both the `read()` and `write()` routines in `nonblocking_disk.C` acquire the lock before issuing a disk command and release it only after the operation has completed. With the macro enabled, only a single thread may interact with the disk at a time; when it is disabled, the system operates in an intentionally unsafe mode suitable for testing and comparison.

Locking Mechanism Overview

To guarantee safe concurrent access to critical sections and avoid race conditions, a locking mechanism is required whenever multiple threads interact with shared resources. This is particularly important for coordinating read and write operations on the disk. The `concurrencyControl` struct provides a lightweight spinlock implementation to manage thread synchronization.

- **Acquiring the Lock:** Before entering the critical section, a thread invokes `concur_control.acquireLock()`. This prevents other threads from accessing the disk simultaneously.
- **Releasing the Lock:** After completing its disk operation, the thread calls `concur_control.releaseLock()` to allow subsequent threads to proceed.
- **Integration:** The `read()` and `write()` methods conditionally wrap their operation sequences with these acquire and release calls when `_THREAD_SYNCHRONIZATION_` is defined.

```
struct concurrencyControl {
    bool flag_control;
    concurrencyControl() : flag_control(false) {}

    bool isSetValid() {
        return flag_control;
    }

    void acquireLock() {
        while (__sync_lock_test_and_set(&flag_control, true)) {
            // Busy wait until lock is acquired (non-blocking)
        }
    }

    void releaseLock() {
        __sync_lock_release(&flag_control); // Release lock atomically
    }
};
```

Figure 6: Implementation of `concurrencyControl` Lock Mechanism

Bonus Option 3: Using Interrupts for Concurrency

To further improve concurrency and eliminate unnecessary CPU busy-waiting, interrupt-based signaling was incorporated into the disk driver. Instead of continuously polling the disk's status register, the system relies on IRQ 14 to notify the kernel when an I/O operation has completed. This allows the CPU to schedule and execute other threads during the waiting period, significantly improving overall efficiency and responsiveness.

Top-End and Bottom-End Separation

The disk I/O workflow is divided into two components:

- **Top-End (Initiation):** When a thread issues a disk `read()` or `write()`, it places itself into the disk's wait queue (if interrupts are enabled) and yields the CPU via `SYSTEM_SCHEDULER->yield()`. This prevents the thread from busy-waiting while the operation is in progress.
- **Bottom-End (Interrupt Handling):** After the disk finishes the requested operation, it triggers IRQ 14. The registered `handle_interrupt()` routine is invoked, which removes the next waiting thread from the disk queue and resumes it by calling `SYSTEM_SCHEDULER->resume()`.

This interrupt-driven design supports concurrent I/O requests from multiple threads while ensuring that only one thread interacts with the disk hardware at any time.

Global Interrupt Enabling for Threads

To ensure that disk-related interrupts are correctly delivered and handled, hardware interrupts must be enabled as soon as a thread begins execution. This is achieved by explicitly invoking `Machine::enable_interrupts()` within the `thread_start()` function. Without enabling interrupts at thread startup, the registered interrupt handler would never be executed, even when the disk generates IRQ 14.

```
static void thread_start() {
    /* This function is used to release the thread for execution in the ready queue. */
    Machine::enable_interrupts();
    /* We need to add code, but it is probably nothing more than enabling interrupts. */
}
```

Figure 7: Interrupts enabled globally when a thread starts execution

Thread Queue and Resumption

The driver maintains a disk-specific thread queue implemented as a FIFO structure. Whenever the disk is not yet ready to complete an operation, the requesting thread is placed into this queue using `enqueue_thread()` and yields the CPU. When the disk later raises an interrupt, the interrupt handler removes the next thread from the queue and resumes its execution.

- This FIFO approach ensures fairness and preserves the ordering of disk-access requests.
- Threads are resumed only once the disk is ready, preventing unnecessary or premature wakeups.

Benefits of This Approach

- Eliminates polling and significantly reduces CPU cycles wasted in busy-wait loops.
- Enhances concurrency by allowing other threads to execute while one thread waits on disk I/O.
- Maintains proper coordination between disk operation completion and thread resumption through the interrupt handler.

```
#ifdef _HANDLE_INTERRUPTS_
  void NonBlockingDisk::handle_interrupt(REGS *_r) {
    Console::puts("[Interrupt] Disk interrupt received. Handling disk interrupt...\n");
    Thread* t = thread_queue.dequeue();
    if (t) {
      Console::puts("[Interrupt] Resuming thread with ID: ");
      Console::puti(t->ThreadId());
      Console::puts("\n");
      SYSTEM_SCHEDULER->resume(t);
    }
  }
#endif
```

Figure 8: Implementation of `handle_interrupt()` and thread resume logic

Testing

To evaluate the correctness and reliability of our `NonBlockingDisk` implementation, we executed three distinct test cases, each reflecting a different configuration of synchronization and interrupt-handling features. All test scenarios used a common kernel setup that initialized four threads: one responsible for performing disk read/write operations, and three additional threads executing computation tasks. The system was built and executed using the following commands:

```
make clean && make
make run
```

Case 1: Basic Disk Operation (No Synchronization, No Interrupt Handling)

Configuration: In this test, both `_HANDLE_INTERRUPTS_` and `_THREAD_SYNCHRONIZATION_` were disabled by commenting them out in `nonblocking_disk.H`.

Expected Behavior:

- Disk operations should run in simple polling mode.
- No interrupt handler should be invoked.
- No locking or synchronization should occur.

Observed Output:

- The console prints "NO DEFAULT INTERRUPT HANDLER REGISTERED," confirming that interrupts were not processed.
- Thread 2 performs disk read and write operations without synchronization, yet all I/O completes correctly.
- All four threads execute with scheduler-driven preemption, showing normal CPU switching behavior.

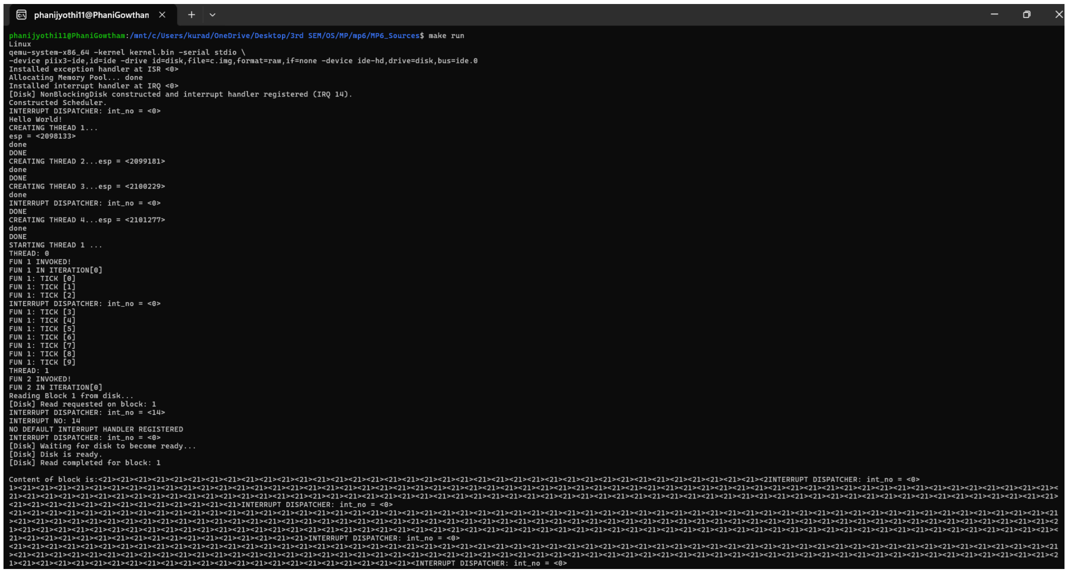

Figure 9: Case 1 Output (Part 1): Initial thread creation and polling-based disk read without synchronization.

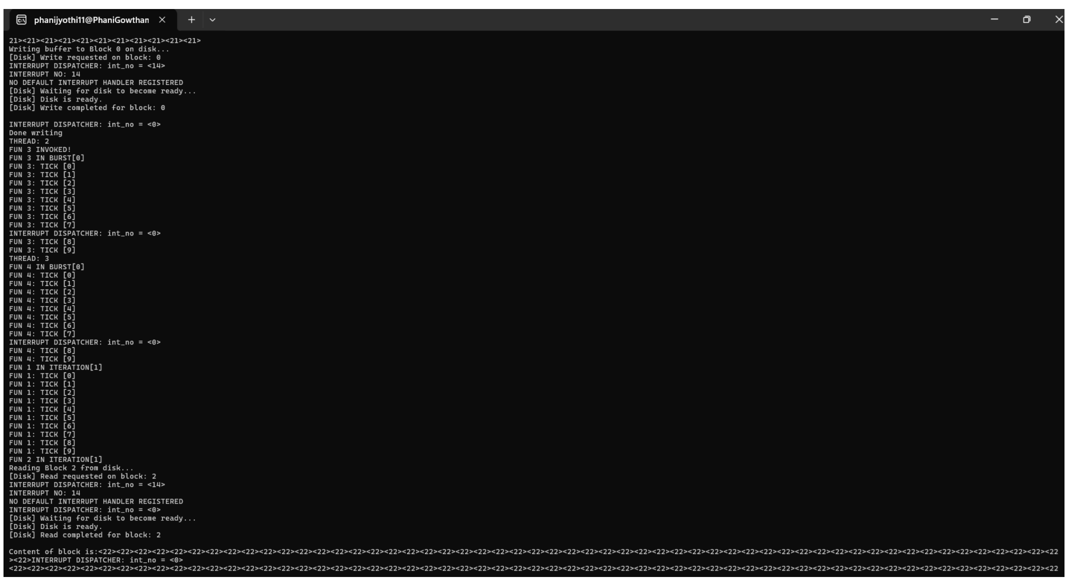

Figure 10: Case 1 Output (Part 2): Continuation of disk operations and thread activity (captured separately due to long output).

Conclusion: The disk operates correctly in polling mode, and the expected fallback behavior is observed when interrupts are not handled.

Case 2 (Bonus Option 1 and 2): With Thread Synchronization Only (No Interrupt Handling)

Configuration: The `_THREAD_SYNCHRONIZATION_` macro was enabled, while `_HANDLE_INTERRUPTS_` remained undefined.

Expected Behavior:

- Disk operations should use the locking mechanism to prevent concurrent access.

- Interrupts should remain unhandled, resulting in the expected fallback warning.
- Threads should yield appropriately while waiting for the disk to become ready.

Observed Output:

- Messages such as `Acquiring lock for read/write` and `Releasing lock after read/write` confirm that the locking mechanism is functioning correctly.
- Interrupts continue to be unhandled, and the fallback notification is printed as expected.
- Threads yield when necessary, demonstrating that the system correctly waits for disk readiness without busy-waiting.

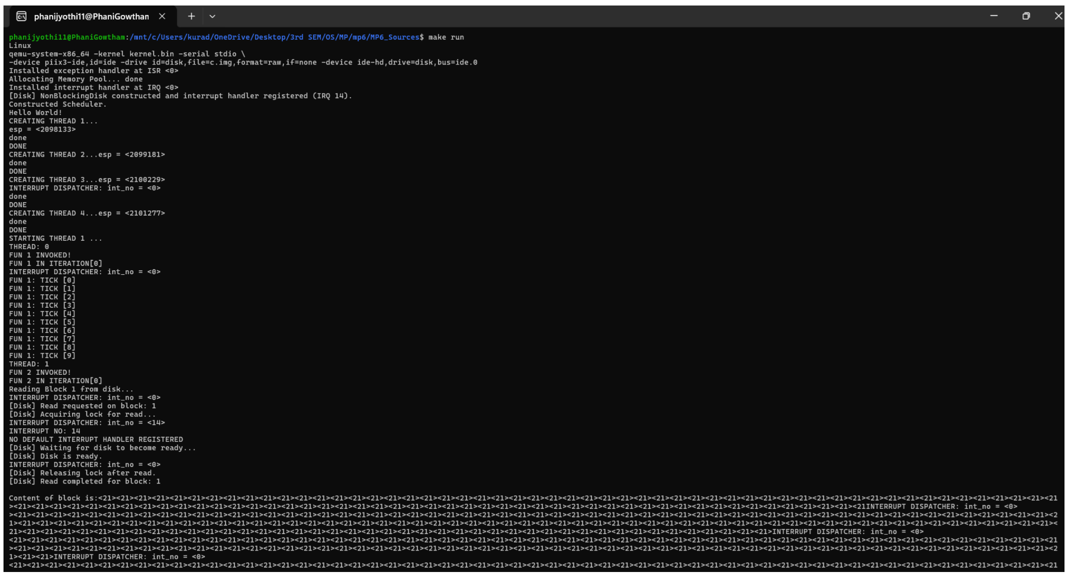

Figure 11: Case 2 Output (Part 1): Disk operations using thread synchronization (locks enabled); interrupts still unhandled.

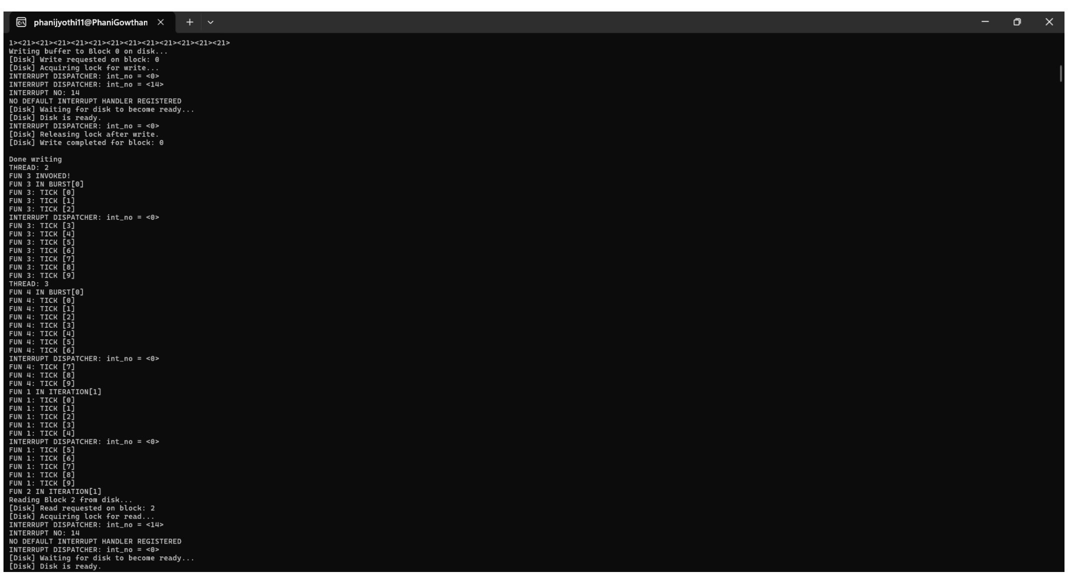

Figure 12: Case 2 Output (Part 2): Continued execution showing correct lock acquisition/release and polling-based disk readiness.

Conclusion: The locking mechanism functions correctly, and polling is used to determine disk readiness when interrupts are disabled.

Case 3 (Bonus Option 3): With Both Synchronization and Interrupt Handling Enabled

Configuration: Both `_HANDLE_INTERRUPTS_` and `_THREAD_SYNCHRONIZATION_` were enabled.

Expected Behavior:

- Disk operations should be serialized using the locking mechanism.
- Disk readiness should be detected through actual hardware interrupts.
- The requesting thread should be enqueued and later resumed when the interrupt fires.

Observed Output:

- Disk read and write operations trigger IRQ 14 exactly as expected.
- Console messages such as `Handling disk interrupt...` and resumption messages involving `SYSTEM_SCHEDULER->resume(t)` confirm correct interrupt handling.
- Locks are consistently acquired and released around disk operations, demonstrating correct synchronization.
- No assertion failures, errors, or unexpected crashes occurred during testing.

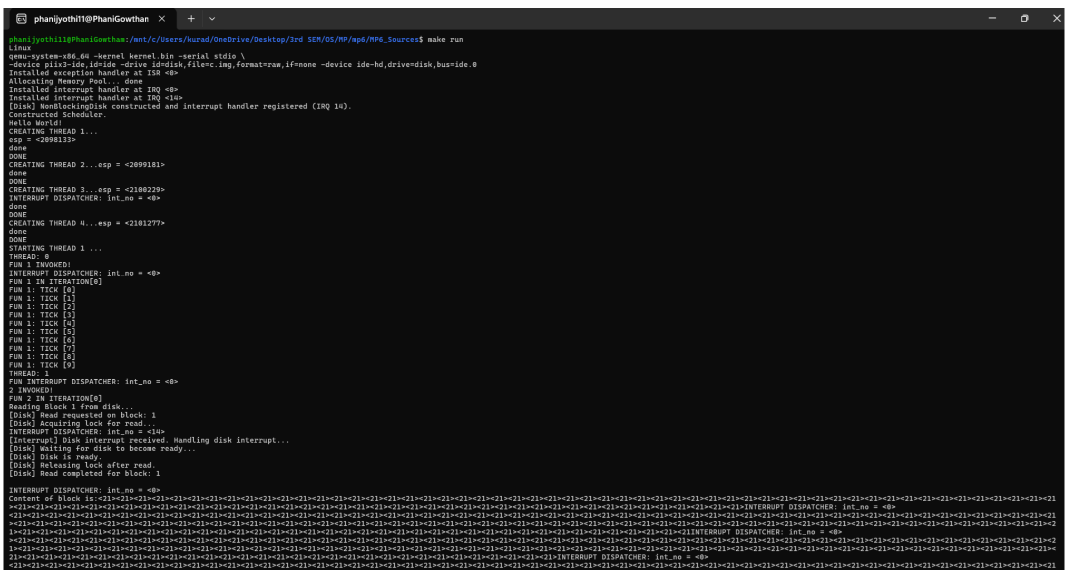

Figure 13: Case 3 Output (Part 1): Disk operations with both synchronization and interrupt handling enabled. Interrupts are correctly triggered and handled.

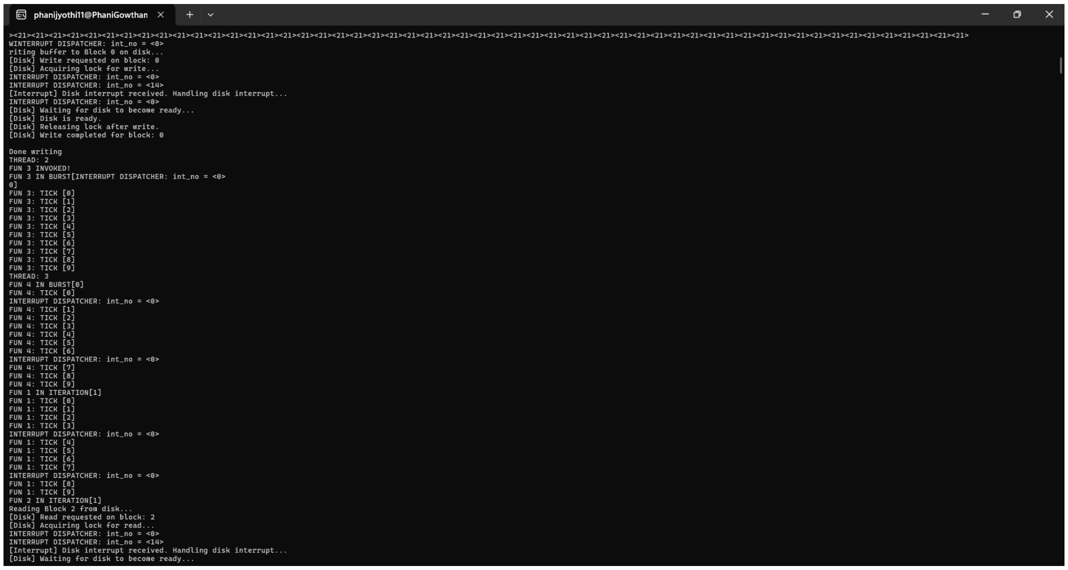

Figure 14: Case 3 Output (Part 2): Continued execution showing synchronized access, interrupt-driven readiness, and thread resumption via scheduler.

Conclusion: Bonus Option 3, which combines interrupt-driven disk handling with synchronization, operates correctly. The system successfully integrates locking, interrupt processing, and scheduler coordination.